Une lettre aux lycéennes et aux lycéens

Comprendre la théorie `full_0`, la programmation compositionnelle et l'ambition de NGeGenBrain

Chères lycéennes, chers lycéens,

Je voudrais vous présenter une idée qui peut sembler compliquée au premier abord, mais qui part d'une question très simple : *comment comprendre le fonctionnement d'un système complexe en l'observant, puis construire un autre système capable de faire une partie du même travail ?*

Cette question est au cur de la théorie appelée `NGeGenBrain_observation_formalisme_full_0`. Elle s'inscrit dans un projet plus large, NGeGenBrain, dont l'ambition est d'étudier les modèles de langage – les systèmes d'intelligence artificielle capables de produire du texte – et de développer des méthodes pour reconstruire des mécanismes fonctionnels à partir d'observations et d'expériences.

Il est important de préciser dès le départ ce que cela signifie : `full_0` fournit un cadre théorique pour organiser cette recherche. Il ne prouve pas, à lui seul, que nous savons déjà reconstruire intégralement un grand modèle de langage réel. C'est une carte pour guider le travail, pas encore la preuve que tout le voyage a été accompli.

1. L'idée de départ : comprendre en observant

Imaginez une boîte fermée. Vous ne pouvez pas voir ce qu'il y a à l'intérieur. Vous pouvez toutefois lui donner des entrées et observer ses réponses. Vous posez une question, la boîte répond ; vous changez la question, la réponse change ; vous modifiez le contexte, et vous observez ce qui se passe.

Un modèle de langage peut être étudié de cette manière. On lui fournit des textes, des consignes ou des contextes, puis on examine ses réponses. On peut aussi comparer ses réactions à des variations contrôlées : changer un mot, ajouter une information, retirer une étape ou poser la même question dans un autre contexte.

Cette démarche ressemble à celle d'une scientifique ou d'un scientifique qui étudie un phénomène : on ne se contente pas de regarder une seule fois. On formule des questions, on réalise des expériences, on note les résultats, on compare les cas et l'on révise ses explications lorsqu'elles ne fonctionnent pas.

`full_0` cherche à donner une forme mathématique à cette démarche. Il distingue notamment :

— ce que l'on observe réellement ;
— les résumés ou signatures que l'on extrait de ces observations ;
— les différences que ces résumés conservent ou perdent ;
— les mécanismes qui pourraient expliquer les résultats ;
— les expériences capables de départager plusieurs explications.

2. Une métaphore : enquêter sur une machine inconnue

Imaginez que vous découvriez un vieux robot dans une salle de classe. Vous ne connaissez ni ses plans ni son programme. Vous pouvez lui donner des instructions simples, observer ses mouvements et noter ses réactions.

Vous pourriez commencer par créer une fiche : ń quand j'appuie sur ce bouton, le robot avance ; quand je change tel réglage, il tourne ż. Cette fiche est utile, mais elle n'est pas le robot lui-même. Elle ne raconte peut-être pas tout : elle peut oublier la vitesse, l'état de la batterie, l'ordre des commandes ou les conditions dans lesquelles l'essai a été effectué.

C'est une bonne image de la différence entre une *observation*, une *empreinte informationnelle* et un *mécanisme*.

— **L'observation**, c'est ce qui s'est passé pendant un essai : l'entrée, le contexte, la sortie, le moment et les conditions.
— **L'empreinte informationnelle**, c'est une description organisée de certains aspects de l'observation. Elle aide à comparer les essais, mais elle peut perdre des détails.
— **Le mécanisme**, c'est une construction exécutable qui produit des comportements. Il doit être testé pour voir s'il explique réellement les observations.

Si deux essais ont la même empreinte, cela ne signifie pas automatiquement qu'ils sont identiques en tout point. Ils peuvent différer sur une information que l'empreinte ne mesure pas. Pour cette raison, la théorie insiste sur une question essentielle : *quelles informations avons-nous conservées, et lesquelles avons-nous laissées de côté ?*

3. Pourquoi une empreinte n'est pas toute l'observation

Une empreinte est comme une carte. Une carte du métro est excellente pour trouver une station, mais elle ne décrit pas chaque rue, chaque arbre ni chaque bâtiment de la ville. Elle est utile précisément parce qu'elle simplifie. Pourtant, on ne peut pas retrouver tous les détails de la ville à partir de cette seule carte.

Il en va de même pour les empreintes d'un système d'intelligence artificielle. Elles peuvent faire ressortir des propriétés utiles, mais elles ne suffisent pas toujours à reconstruire la totalité du comportement observé.

Pour reconstruire exactement une observation à partir d'une empreinte, il faut que cette empreinte conserve assez d'information pour distinguer toutes les observations que l'on veut différencier. Si plusieurs observations différentes conduisent à la même empreinte, l'empreinte seule ne permet pas de savoir laquelle a eu lieu.

Il faut donc parfois conserver, en plus de l'empreinte :

— les variantes de la situation ;
— le contexte et l'historique ;
— la sonde utilisée, c'est-à-dire la question ou l'expérience ;
— les incertitudes et les limites de la mesure ;
— les traces des interventions réalisées.

Le traité formalise cette distinction et donne des conditions précises sous lesquelles une reconstruction exacte est possible. Il rappelle aussi qu'une propriété dite ń invariante ż ne l'est pas dans l'absolu : il faut préciser quelles transformations ou quels changements de contexte sont autorisés, puis vérifier que la propriété reste effectivement la même sous ces changements.

4. Pourquoi observer à plusieurs échelles et dans le temps ?

Une photographie peut montrer la position d'un ballon, mais pas à elle seule sa trajectoire. Pour comprendre le mouvement, il faut plusieurs images prises à différents moments. De la même façon, une seule réponse d'un modèle de langage ne suffit généralement pas pour comprendre son comportement.

On peut étudier :

— un détail précis, comme la réaction à un mot ;
— une réponse complète ;

— une suite de questions et de réponses ;
— un ensemble de tâches différentes ;
— les changements de comportement lorsque le contexte ou les consignes changent.

La théorie `full_0` propose de penser ces observations à plusieurs échelles et sur plusieurs instants. Une observation plus riche peut révéler des différences qu'une observation plus simple ne permettait pas de voir.

Cela ne veut pas dire qu'il faut tout mesurer sans limite. Il faut choisir les expériences en fonction de la question posée, conserver les informations pertinentes et noter les limites de ce que l'on peut conclure.

5. Le rôle décisif de la programmation compositionnelle

Voici une idée particulièrement importante pour NGeGenBrain : la **programmation compositionnelle**.

Le mot ń compositionnelle ż signifie que l'on construit un système complexe en assemblant des éléments plus petits, suivant des règles précises. C'est comparable à des briques de construction : chaque brique a une forme et un rôle ; les règles d'assemblage permettent de fabriquer une structure plus grande.

On retrouve cette idée dans de nombreux domaines :

— en mathématiques, où des fonctions peuvent être composées ;
— en musique, où des motifs peuvent former des phrases, puis des morceaux ;
— en ingénierie, où des modules testés sont assemblés en systèmes ;
— en informatique, où des fonctions et des composants coopèrent grâce à des interfaces définies.

Pour NGeGenBrain, cette approche pourrait changer la difficulté du problème. Au lieu de traiter le modèle entier comme une boîte unique et indéchiffrable, le projet chercherait à identifier des comportements ou des composants candidats, à préciser leurs entrées et sorties, à les tester séparément, puis à étudier comment les assembler.

On peut représenter cette ambition par la chaîne suivante :

observer
↓
décomposer
↓
reconstruire des composants
↓
composer
↓
tester le système assemblé

Mais la métaphore des briques a une limite importante. Des briques compatibles ne garantissent pas à elles seules que le bâtiment sera solide. De même, deux composants qui fonctionnent séparément peuvent échouer lorsqu'ils sont assemblés. Il faut donc définir leurs interfaces, leurs préconditions, les effets qu'ils produisent et les garanties que leur composition préserve.

La programmation compositionnelle n'est donc pas simplement une façon d'écrire du code plus propre. Dans la vision de NGeGenBrain, elle pourrait devenir le principe d'organisation qui relie l'observation, la reconstruction, la vérification et la construction progressive d'un mécanisme.

6. Comment `full_0`, la composition et NGeGenBrain se complètent

Ces trois éléments ont des rôles différents mais complémentaires.

`full_0` : le cadre de raisonnement
Il formalise la manière de parler des observations, des empreintes, des pertes d'information, des mécanismes candidats, de la récursion et de la réobservation. Il aide à préciser ce que l'on sait, ce que l'on suppose et ce qu'il reste à démontrer.

La programmation compositionnelle : la méthode d'assemblage
Elle fournit le principe permettant de construire des systèmes à partir de composants, avec des interfaces et des règles de composition explicites. Elle peut permettre de tester localement certaines parties et de réutiliser les composants qui ont déjà été caractérisés.

NGeGenBrain : le système de recherche et de construction
Il représente le projet qui mettrait ces idées en pratique : collecter les observations, organiser les expériences, conserver les preuves et les traces, proposer des composants, les assembler, exécuter les candidats et comparer leurs résultats avec ceux du modèle étudié.

Une vue d'ensemble possible est :

observations du modèle source
↓
empreintes, variantes et traces
↓
hypothèses de composants
↓
composants exécutables et règles de composition
↓
système candidat
↓
tests comparatifs et nouvelles observations
↓
correction, raffinement ou rejet

Cette chaîne est une vision de travail : chaque flèche doit devenir une procédure précise, testable et documentée. Il ne suffit pas de donner un nom à chaque étape. Il faut définir les données qu'elle reçoit, ce qu'elle produit, les conditions dans lesquelles elle est valide et la façon dont on détecte ses erreurs.

7. À quoi cela pourrait-il servir ?

Cette approche pourrait être utile dans plusieurs situations, à condition que les méthodes soient effectivement développées et validées.

1. **Comprendre les comportements d'un modèle.** Comparer ses réponses à des expériences contrôlées et découvrir quelles modifications changent réellement son comportement.
2. **Construire des systèmes plus petits.** Reproduire certaines capacités ciblées d'un modèle sans prétendre reproduire tout le modèle original.
3. **Tester des hypothèses.** Mettre en concurrence plusieurs explications possibles et concevoir des expériences qui permettent de les distinguer.
4. **Réutiliser des composants.** Si des composants ont des interfaces claires et des propriétés vérifiées, ils pourraient être réutilisés dans d'autres systèmes.

5. **Améliorer la traçabilité.** Relier chaque choix de reconstruction aux observations, aux expériences et aux tests qui le justifient.
6. **Détecter les limites.** Repérer les tâches où le système reconstruit échoue, les informations qui lui manquent et les situations où il ne faut pas lui faire confiance.

Par exemple, imaginons que le modèle source réussisse une tâche de classement de textes. NGeGenBrain pourrait chercher à reconstruire un mécanisme qui reproduit cette capacité sur un domaine clairement défini. Il faudrait ensuite tester le mécanisme sur de nouveaux exemples, y compris des cas difficiles. S'il échoue, l'échec devient une information : il peut montrer qu'un composant est insuffisant, que la composition est incorrecte ou que les expériences initiales étaient trop limitées.

8. Quelles pourraient être les conséquences ?

Si cette démarche aboutit, elle pourrait rendre l'étude de certains systèmes d'IA plus méthodique, plus modulaire et plus vérifiable. Elle pourrait faciliter la comparaison de modèles, la création de systèmes spécialisés, l'analyse de leurs limites et la reproduction contrôlée de certaines capacités.

Elle pourrait aussi aider à mieux distinguer trois affirmations très différentes :

— ń le système candidat donne quelques réponses qui ressemblent à celles du modèle source ż ;
— ń le système candidat reproduit un ensemble défini de comportements sur un domaine testé ż ;
— ń le système candidat reproduit le comportement du modèle source dans tous les cas pertinents ż.

La première affirmation est faible ; la deuxième est plus solide si les tests sont bien conçus ; la troisième est extrêmement exigeante et ne doit jamais être annoncée sans preuves adaptées. Réussir quelques tests ne démontre pas une équivalence universelle.

Il existe aussi des défis pratiques : le coût des expériences, l'accès au modèle source, la difficulté d'observer certaines informations internes, le choix des composants, la gestion des erreurs et la possibilité que plusieurs mécanismes différents expliquent les mêmes observations. La rigueur consiste à rendre ces difficultés visibles plutôt qu'à les cacher.

9. La récursion : apprendre de ses propres erreurs

Le projet ne devrait pas suivre une seule fois la chaîne observation–reconstruction–test puis s'arrêter. Il devrait pouvoir recommencer en utilisant les résultats obtenus.

C'est l'idée de la récursion : construire une première version, l'observer, repérer les écarts, revenir sur les composants concernés, les modifier et tester à nouveau. On peut aussi décomposer un composant en sous-composants si les expériences montrent qu'il est trop grossier pour expliquer les résultats.

Cette boucle ressemble à l'apprentissage d'un instrument de musique. On joue un morceau, on écoute ce qui ne va pas, on travaille une mesure, puis on rejoue l'ensemble. Chaque répétition doit apporter une information ou une amélioration mesurable ; répéter sans critère de réussite ne garantit aucun progrès.

NGeGenBrain devra donc définir des critères d'arrêt et des budgets : combien d'expériences réaliser, quand approfondir un composant, quand accepter une approximation et quand conclure que les données ne permettent pas de trancher. La récursion ne garantit pas automatiquement la convergence.

10. La vision d'avenir

La vision à long terme est de faire de NGeGenBrain un environnement de recherche capable de passer progressivement de l'observation d'un système à la construction de mécanismes candidats, puis à leur vérification expérimentale.

Dans une version ambitieuse, un tel environnement pourrait :

1. choisir des expériences informatives plutôt que multiplier les essais au hasard ;
2. comparer les résultats et conserver les preuves de chaque conclusion ;
3. proposer des décompositions en composants, sans confondre ces décompositions avec une certitude sur l'architecture interne réelle ;
4. synthétiser ou sélectionner des composants exécutables ;
5. composer ces composants selon des règles explicites ;
6. tester le système assemblé contre le modèle source sur des tâches nouvelles ;
7. localiser les divergences, réviser les hypothèses et recommencer ;
8. publier un bilan précis : ce qui fonctionne, ce qui échoue, ce qui est prouvé et ce qui reste inconnu.

À terme, le but ne serait pas simplement de produire une copie superficielle des réponses d'un modèle. Il serait de construire des mécanismes dont le comportement est compris dans un domaine donné, dont les composants peuvent être examinés et dont les limites sont documentées.

Il faut cependant rester honnête : cette vision est un objectif de recherche, pas une capacité déjà démontrée par le seul document `full_0`. Pour s'en rapprocher, il faudra transformer les définitions et les théorèmes en algorithmes exécutables, mener des expériences reproductibles et publier les résultats, y compris les échecs.

11. Pourquoi cela concerne aussi les lycéennes et les lycéens

Vous n'avez pas besoin d'être déjà spécialistes en intelligence artificielle pour comprendre le cur de cette démarche. Elle fait appel à des idées que vous pouvez commencer à pratiquer :

— **Les mathématiques**, pour définir précisément les objets et démontrer les propriétés annoncées ;
— **L'informatique**, pour écrire les composants, les interfaces et les expériences ;
— **Les sciences expérimentales**, pour formuler des hypothèses et les confronter aux résultats ;
— **La logique et l'esprit critique**, pour distinguer une observation, une explication possible et une preuve ;
— **Le travail collectif**, car un projet complexe exige plusieurs compétences et une documentation compréhensible.

Vous pouvez commencer avec un système très simple : un programme qui classe des phrases, un petit jeu, un robot virtuel ou un modèle de langage accessible. Choisissez quelques entrées, formulez des hypothèses, notez les sorties, changez une seule condition à la fois, puis essayez de construire un petit programme qui reproduit le comportement observé. Comparez-le sur des exemples qu'il n'a jamais vus. C'est une première expérience de la démarche, à une échelle adaptée.

En conclusion

La théorie `full_0` pose une question profonde : comment passer d'observations organisées à des hypothèses de mécanismes, puis tester si ces mécanismes expliquent réellement ce que nous avons observé ?

La programmation compositionnelle apporte une possibilité majeure : au lieu de viser directement un système immense et indivisible, construire et vérifier des éléments plus petits, puis étudier les règles qui permettent de les assembler. NGeGenBrain est le projet dans lequel ces idées pourraient être réunies en une méthode et en un outil de recherche exécutables.

La métaphore finale est celle d'une équipe qui étudie un instrument complexe. `full_0` aide à organiser les observations et le raisonnement ; la programmation compositionnelle aide à assembler des pièces ; NGeGenBrain serait l'atelier où l'on construit, mesure, teste, corrige et documente les prototypes.

Le succès ne se mesurera pas au nombre de pages, de formules ou de composants, mais à la qualité des expériences, à la solidité des démonstrations et à la capacité de dire clairement ce que le système reconstruit sait faire – et ce qu'il ne sait pas faire encore.

La grande ambition n'est pas de prétendre tout comprendre d'un coup, mais de rendre chaque étape de la compréhension vérifiable, réutilisable et améliorable.

Avec confiance dans votre curiosité et votre esprit critique,

L'équipe du projet NGeGenBrain